

Chapter 9: Deep Learning

Outline

Introduction

Convolutional Neural Networks

Computer Vision

Recurrent models and attention

Graph Neural Networks

Conclusions

9.1 Introduction

- ▶ Deep learning (DL) is a subset of ML methods in which the model is also provided with the means to extract or engineer meaningful features directly from data.
- ▶ Since the feature extraction is now part of the model, it can be trained automatically.
- ▶ This is especially useful in applications where the structure of the data, that is the relative position between points, is important such as images, audio, text, and time series data.

9.2 Convolutional Neural Networks

- ▶ Convolutional neural networks (CNNs) are a class of ANNs designed to be effective at processing images.
- ▶ CNNs apply learnable filters to local regions of the images, producing features that represent those regions.
- ▶ The main operations in CNNs are convolution, pooling, and down/up-sampling.

CNN Operations - Convolution

- ▶ Convolution is the application of a filter to an input that results in an activation.
- ▶ A filter applied on an $M \times N$ matrix is defined in terms of size $m \times n$, where $m < M$ and $n < N$, and a *stride* that determines how much the filter moves before being applied to another portion of the output.

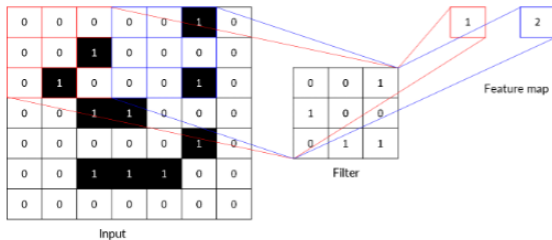


Figure: Convolution operation.

CNN Operations - Convolution and Receptive Field

- ▶ For a certain filter size and stride, some regions of the input will never fall together in the same convolution operation, and their relationships will not be considered by the model.
- ▶ This restriction produces a so-called *receptive field*.
- ▶ Stacking layers expands receptive field and captures hierarchical features.

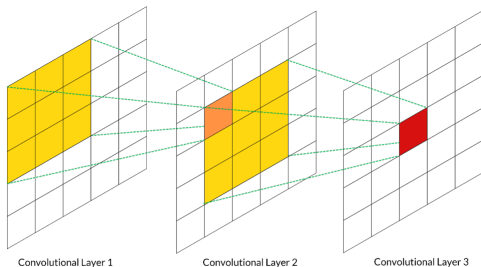


Figure: Receptive field.

CNN operations - Pooling

- Pooling reduces the size of the feature map and with it the number of parameters to be trained in the CNN.

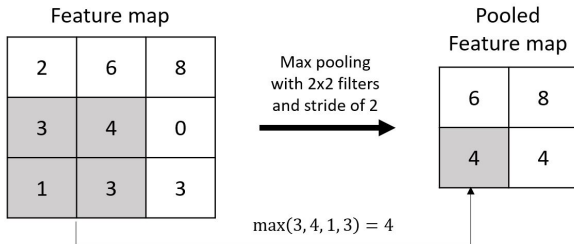


Figure: Max-pooling.

CNN operations - Down Sampling and Up Sampling

- ▶ Down sampling reduces the resolution of an image, and can be done via pooling.
- ▶ Up sampling increases the resolution and can be achieved with transpose convolution.

CNN operations - Combined

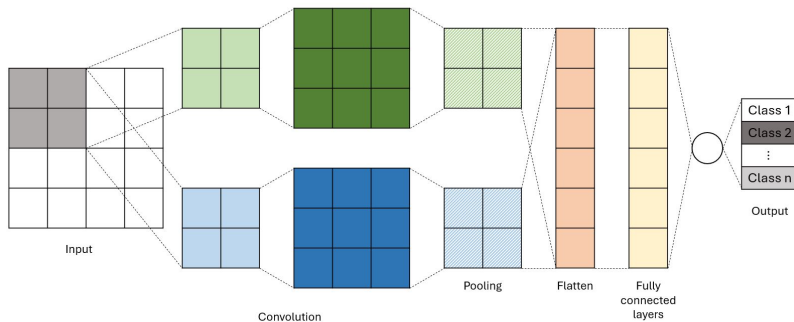


Figure: Convolution and pooling combined in a CNN.

9.3 Computer Vision

- ▶ Applications of CNN for image processing are often group in under the umbrella of *computer vision* (CV).
- ▶ CV tasks include
 - ▶ Object classification
 - ▶ Object localization
 - ▶ Object detection
 - ▶ Image segmentation

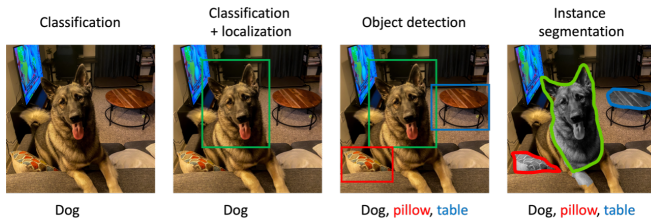


Figure: Tasks in computer vision.

Notable architectures

- ▶ CNN developments kicked-off the era of large models in AI/ML.
- ▶ Advances in architectures and training methods enabled models with millions of parameters such as:
 - ▶ VGG: deep stacks of 3x3 convs; many parameters.
 - ▶ ResNet: residual connections enable very deep networks.
 - ▶ U-Net/FCN: encoder-decoder designs for segmentation.
- ▶ Transfer learning and related ideas enable the adaptation of models trained on millions of images for specific tasks.

9.4 Recurrent Neural Networks

- ▶ In image analysis, for each record we not only care about the value in each pixel but also where in the input matrix it is located.
- ▶ What about relationships between records?
- ▶ If we do not assume that each record is independent, and we can distribute these records over time or space, then we have sequence data.
- ▶ Text, audio signals, and time series are all examples of sequence data.
- ▶ For sequence data we are interested in predicting what will happen next.
- ▶ Recursive neural networks (RNN) are a class of ANNs designed to be effective as processing sequence data.

9.4 Recurrent Neural Networks - Basis

- RNNs use *previous* outputs of the network units, called *hidden states*, h_{t-1} , as inputs for the *next* prediction, y_t .

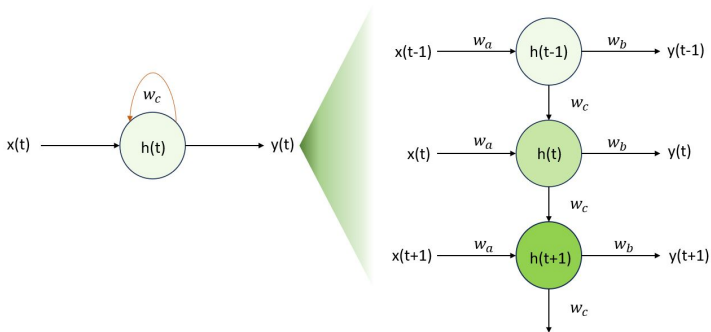


Figure: Basic structure of RNNs.

9.4 Recurrent Neural Networks - Long Short-Term Memory

- ▶ In the RNN described before, the model *remembers* only one step before the current prediction.
- ▶ The previous prediction is in turn informed by its predecessor, so some past information is carried out.
- ▶ However, the influence of previous steps vanishes quickly for bigger models.
- ▶ The question then becomes *how can we make the model remember more?* and *how far back should it remember?*.

9.4 Recurrent Neural Networks - Long Short-Term Memory

- ▶ Long Short-Term Memory (LSTM) networks address this issue by modifying the hidden state based on three sources of information called *gates*: forget gate, $f_{t,s}$, input gate $i_{t,s}$, and output gate $o_{t,s}$.
- ▶ The new hidden state is
$$h_{t,s} = o_{t,s} \times \tanh(c_{t,s}),$$
where $c_{t,s} = f_{t,s} \times c_{t-1,s} + i_{t,s} \times \tilde{c}_{t,s}$,and $\tilde{c}_{t,s} = \tanh(W_c x_{t,s} + U_c h_{t-1} + b_c)$

9.5 Attention and Transformers

- ▶ In RNNs, even with LSTMs, the more recent a previous step the more it influences the output.
- ▶ In some cases, however the most important information, might not be at the end of the sequence.
- ▶ Now we are not only interested in how far back we should remember but also what exactly should we remember.

9.5 Attention and Transformers

- ▶ *Attention* is an operation that allows the computer to derive relationships between blocks of data at *any position* in the sequence.
- ▶ Mathematically

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (1)$$

- ▶ Here Q , K , and V the query, key, and value matrices learned during training, and d_k is the dimension of the keys.

9.5 Attention and Transformers

- *Transformers* are a class of ANNs that use attention operations to better capture complex sequences.

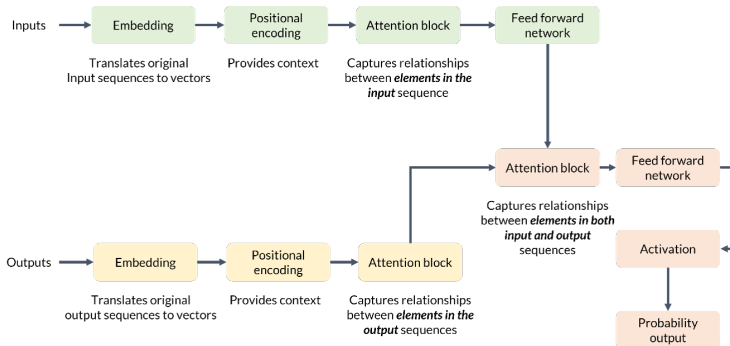


Figure: Basic structure of transformers.

9.6 Graph Neural Networks

- ▶ Along with images and sequence data, graphs are a type of data in which the intrinsic structure of the records carries important information.
- ▶ Molecular structure are an example of graph data. The nature of the atoms (nodes) and bonds (edges) influences the properties of the molecule.
- ▶ Graph neural networks (GNNs) are a class of ANNs designed to exploit graph representations.

9.6 Graph Neural Networks

- ▶ The fundamental idea behind GNNs is to extract features for each node in a graph by aggregating information from neighboring nodes.
- ▶ Mathematically,

$$h_v^{(l+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v)} f \left(h_u^{(l)}, h_v^{(l)}, e_{uv} \right) \right) \quad (2)$$

- ▶ Here, $h_v^{(l)}$ is the representation of node v at layer l , $\mathcal{N}(v)$ denotes the neighbors of v , f is a differentiable function that aggregates information from neighboring nodes, and e_{uv} represents the edge features between nodes u and v .

9.7 Conclusions

- ▶ We discussed deep learning with an emphasis on efficient data representations, focusing on convolutional neural networks (CNNs), recurrent neural networks (RNNs), and graph neural networks (GNNs).
- ▶ We reviewed the historical development and core ideas of CNNs, and discussed their central role in computer vision tasks.
- ▶ We examined RNNs for sequential data modeling, their limitations and extensions (LSTM), as well as attention and transformers.
- ▶ We introduced GNNs as a framework for learning from graph data, highlighting their neighborhood-based representation updates.